

SECTION A: ANSWERS AND MARKING GUIDE

1. a. 10
2. c. Overridden method must have same return type
3. c. An abstract class can have constructors
4. a. X
5. c. protected
6. d. Both b and c
7. b. true and then 123
8. c. Thread cannot restart after termination
9. c. this
10. b. Does not release lock
11. b. Constants are static final
12. a. Yes, private
13. a. Connection
14. b. ObjectOutputStream
15. b. BufferedReader
16. b. IOException
17. a. Parent class object
18. d. Set
19. b. Downcasting incompatible types
20. b. TreeSet
21. b. 11 10
22. b. Polymorphism
23. a. Declaring class fields as private and providing public getters/setters
24. d. `^[^0-9]+$`

25. b. Overriding requires same method signature, overloading does not

SECTION B: ANSWERS AND MARKING GUIDE

Question 26: Differentiate between Abstract Class and Interface

An abstract class is a class that can contain both abstract and concrete methods. It can have instance variables and constructors. An interface is a completely abstract structure used to achieve multiple inheritance. A class can extend only one abstract class but can implement multiple interfaces.

Question 27:

a) A custom exception in Java is created by extending the Exception class. The exception is thrown using the throw keyword and handled using try-catch blocks.

b) Example code:

```
class MyException extends Exception { }  
throw new MyException();
```

Question 28:

Generics were introduced in Java to provide type safety and reduce runtime errors. Before generics, collections stored objects as Object type, leading to ClassCastException. Generics solve this by enforcing type checking at compile time.

Question 29:

Java achieves multiple inheritance using interfaces. A class can implement multiple interfaces and provide implementations for their methods.

Question 30:

The throw keyword is used to explicitly throw an exception in a program. The throws keyword is used in method declaration to indicate exceptions that may occur.

SECTION B: MARKING GUIDE

Question 26: Abstract Class vs Interface (5 Marks)

Correct definition of abstract class – 1 mark

Correct definition of interface – 1 mark

Relevant Java example – 3 marks

Question 27: Custom Exception (5 Marks)

a) 2 marks

b) 3 marks

Question 28: Generics in Java (5 Marks)

Explanation of why generics were introduced – 2 marks

Problem before generics (ClassCastException) – 2 marks

Clear example showing solution using generics – 1 mark

Question 29: Multiple Inheritance in Java (5 Marks)

Explanation that Java uses interfaces – 2 marks

Correct interface declaration – 1 mark

Class implementing multiple interfaces – 1 mark

Valid example usage – 1 mark

Question 30: throw vs throws (5 Marks)

Correct explanation of throw – 2 marks

Correct explanation of throws – 2 marks

Appropriate code examples – 1 mark

SECTION C: ANSWERS AND MARKING GUIDE

```
31. class BankAccount {  
    private double balance;  
  
    public BankAccount(double balance) {  
        this.balance = balance;  
    }  
  
    // synchronized deposit method  
    public synchronized void deposit(double amount) {  
        balance += amount;  
        System.out.println(Thread.currentThread().getName()  
            + " deposited: " + amount + " | Balance: " + balance);  
    }  
  
    // synchronized withdraw method  
    public synchronized void withdraw(double amount) {  
        if (balance >= amount) {  
            balance -= amount;  
            System.out.println(Thread.currentThread().getName()  
                + " withdrew: " + amount + " | Balance: " + balance);  
        } else {  
            System.out.println(Thread.currentThread().getName()  
                + " insufficient balance");  
        }  
    }  
}
```

```
    public double getBalance() {  
        return balance;  
    }  
}
```

```
class DepositThread extends Thread {  
    private BankAccount account;  
  
    public DepositThread(BankAccount account) {  
        this.account = account;  
    }  
  
    public void run() {  
        account.deposit(500);  
    }  
}
```

```
class WithdrawThread extends Thread {  
    private BankAccount account;  
  
    public WithdrawThread(BankAccount account) {  
        this.account = account;  
    }  
  
    public void run() {  
        account.withdraw(300);  
    }  
}
```

```

    }
}

public class BankTest {

    public static void main(String[] args) {

        BankAccount account = new BankAccount(1000);

        Thread t1 = new DepositThread(account);
        Thread t2 = new WithdrawThread(account);
        Thread t3 = new WithdrawThread(account);

        t1.start();
        t2.start();
        t3.start();

    }
}

```

Marking Scheme: Thread-Safe Bank Account (15 Marks)

Component	Description	Marks
BankAccount class	Proper definition with private balance attribute	3
Deposit & withdrawal methods	Correct logic for deposit and withdraw operations	3

Thread safety	Correct use of synchronized keyword to protect shared resource	4
Multithreading implementation	Creation and use of Thread or Runnable class	2
Use of two threads in main	Two threads created and started on the same BankAccount object	2

```
import java.util.*;
```

```
// Book class with encapsulation
```

```
class Book {
```

```
    private int bookId;
```

```
    private String title;
```

```
    private String author;
```

```
    private int yearPublished;
```

```
    public Book(int bookId, String title, String author, int yearPublished) {
```

```
        this.bookId = bookId;
```

```
        this.title = title;
```

```
        this.author = author;
```

```
        this.yearPublished = yearPublished;
```

```
    }
```

```
// Getters and setters
```

```
public int getBookId() {  
    return bookId;  
}
```

```
public void setBookId(int bookId) {  
    this.bookId = bookId;  
}
```

```
public String getTitle() {  
    return title;  
}
```

```
public String getAuthor() {  
    return author;  
}
```

```
public int getYearPublished() {  
    return yearPublished;  
}  
}
```

// Comparator for sorting by author

```
class AuthorComparator implements Comparator<Book> {  
    public int compare(Book b1, Book b2) {  
        return b1.getAuthor().compareTo(b2.getAuthor());  
    }  
}
```



```
}
```

```
// Comparator for sorting by title
```

```
class TitleComparator implements Comparator<Book> {
```

```
    public int compare(Book b1, Book b2) {
```

```
        return b1.getTitle().compareTo(b2.getTitle());
```

```
    }
```

```
}
```

```
// Comparator for sorting by year published
```

```
class YearComparator implements Comparator<Book> {
```

```
    public int compare(Book b1, Book b2) {
```

```
        return Integer.compare(b1.getYearPublished(), b2.getYearPublished());
```

```
    }
```

```
}
```

```
public class LibraryTest {
```

```
    public static void main(String[] args) {
```

```
        // Store books in a collection
```

```
        List<Book> books = new ArrayList<>();
```

```
        books.add(new Book(1, "Java Basics", "James", 2018));
```

```
        books.add(new Book(2, "Data Structures", "Alice", 2020));
```

```
        books.add(new Book(3, "OOP Concepts", "Brian", 2015));
```

```

// Sort by Author
Collections.sort(books, new AuthorComparator());

// Sort by Title
Collections.sort(books, new TitleComparator());

// Sort by Year Published
Collections.sort(books, new YearComparator());

// Display books
for (Book b : books) {
    System.out.println(
        b.getBookId() + " " +
        b.getTitle() + " " +
        b.getAuthor() + " " +
        b.getYearPublished()
    );
}
}
}

```

Marking Scheme: Library System using Java (15 Marks)

Component	Description	Marks
-----------	-------------	-------

Book class design	Book class with bookId, title, author, yearPublished	3
Encapsulation	Proper use of private attributes with getters/setters	3
Use of Collection	Books stored in a Collection (e.g., ArrayList)	2
Comparator implementation	Correct Comparator classes for Author, Title, and Year	4
Sorting & demonstration	Correct sorting and output in main method	3

Marking Scheme: Thread-Safe Bank Account (15 Marks)

Component	Description	Marks
BankAccount class	Proper definition with private balance attribute	3
Deposit & withdrawal methods	Correct logic for deposit and withdraw operations	3
Thread safety	Correct use of synchronized keyword to protect shared resource	4
Multithreading implementation	Creation and use of Thread or Runnable class	2

Use of two threads in main	Two threads created and started on the same BankAccount object	2
----------------------------	--	---